

# Shell Scripting Lecture Notes

Tanvir Ahammad

May 28, 2024

## 1 Introduction to Shell Scripting

### 1.1 What is a Shell?

- **Shell:** A command-line interpreter that provides a user interface for the Unix operating system.
- **Types of Shells:**
  - **Bourne Shell (sh)**
  - **C Shell (csh)**
  - **Korn Shell (ksh)**
  - **Bourne Again Shell (bash)** - Most common in Linux systems

### 1.2 What is Shell Scripting?

- A script written for the shell, or command line interpreter of an operating system.
- It contains a series of commands that the shell executes.

### 1.3 Why Use Shell Scripting?

- Automate repetitive tasks
- Easily manipulate files and directories
- Integrate and execute multiple commands

## 2 Getting Started with Shell Scripting

### 2.1 Writing Your First Script

1. Open a text editor (e.g., `nano`, `vim`, `gedit`).
2. Write the script:

```
#!/bin/bash
echo "Hello, World!"
```

3. Save the file with a `.sh` extension, e.g., `hello.sh`.

4. Make the script executable:

```
chmod +x hello.sh
```

5. Run the script:

```
./hello.sh
```

## 2.2 Basic Shell Commands

- **File Operations:** `ls`, `cp`, `mv`, `rm`, `touch`, `mkdir`, `rmdir`
- **Text Operations:** `cat`, `echo`, `grep`, `awk`, `sed`
- **System Operations:** `ps`, `kill`, `df`, `du`, `top`

## 3 Shell Scripting Basics

### 3.1 Variables

- Define a variable:

```
NAME="John"
```

- Access a variable:

```
echo $NAME
```

### 3.2 Comments

- Single-line comment:

```
# This is a comment
```

- Multi-line comment:

```
: '  
This is a  
multi-line comment  
,
```

### 3.3 Input and Output

- Reading User Input:

```
echo "Enter your name:"  
read NAME  
echo "Hello, $NAME"
```

## 4 Control Structures

### 4.1 Conditional Statements

- If Statement:

```
if [ condition ]; then  
    # commands  
fi
```

- If-Else Statement:

```
if [ condition ]; then  
    # commands  
else  
    # other commands  
fi
```

- If-Elif-Else Statement:

```
if [ condition1 ]; then  
    # commands  
elif [ condition2 ]; then  
    # commands  
else  
    # commands
```

```
fi
```

## 4.2 Loops

- For Loop:

```
for i in 1 2 3 4 5; do
    echo "Welcome $i times"
done
```

- While Loop:

```
count=1
while [ $count -le 5 ]; do
    echo "Count is $count"
    ((count++))
done
```

- Until Loop:

```
count=1
until [ $count -gt 5 ]; do
    echo "Count is $count"
    ((count++))
done
```

## 5 Functions

### 5.1 Defining and Calling Functions

- Define a function:

```
function name() {
    # commands
}
```

- Call a function:

```
name
```

## 5.2 Passing Arguments to Functions

- Define a function with parameters:

```
function greet() {  
    echo "Hello, $1"  
}
```

- Call the function with arguments:

```
greet "John"
```

## 6 Advanced Topics

### 6.1 Script Arguments

- Accessing script arguments:

```
echo "Script name: $0"  
echo "First argument: $1"  
echo "Second argument: $2"
```

### 6.2 Arrays

- Define an array:

```
NAMES=("Alice" "Bob" "Charlie")
```

- Access array elements:

```
echo ${NAMES[0]} # Outputs: Alice
```

### 6.3 String Manipulation

- Concatenate strings:

```
FULL_NAME="$FIRST_NAME $LAST_NAME"
```

- Get string length:

```
echo ${#STRING}
```

## 6.4 File Operations in Scripts

- Check if a file exists:

```
if [ -e filename ]; then
    echo "File exists"
fi
```

## 6.5 Error Handling

- Exit on error:

```
set -e
```

## 6.6 Debugging

- Enable debugging:

```
set -x
```

# 7 Practical Examples

## 7.1 Backup Script

```
#!/bin/bash
# Backup script

SOURCE="/path/to/source"
DESTINATION="/path/to/destination"
DATE=$(date +%Y-%m-%d)

tar -czvf $DESTINATION/backup-$DATE.tar.gz $SOURCE
```

## 7.2 User Creation Script

```
#!/bin/bash
# User creation script

if [ $# -ne 1 ]; then
    echo "Usage: $0 username"
    exit 1
fi

USERNAME=$1
useradd $USERNAME
echo "User $USERNAME created."
```

## 8 Summary

- Shell scripting is a powerful tool for automating tasks.
- Basic syntax includes defining variables, using control structures, and creating functions.
- Advanced topics include handling script arguments, arrays, string manipulation, and file operations.
- Practical examples demonstrate how to apply shell scripting in real-world scenarios.

## 9 Exercises

1. Write a script to display the current date and time.
2. Create a script that takes a filename as an argument and displays its contents.
3. Write a script to check if a directory exists, and if not, create it.
4. Create a script that counts the number of files in a directory.
5. Write a backup script that compresses a specified directory and stores it in a backup folder.

## 10 Easy Problems

### 10.1 Problem 1: Display Current Date and Time

**Problem:** Write a script to display the current date and time.

**Solution:**

```
#!/bin/bash
# Display current date and time
echo "Current date and time: $(date)"
```

## 10.2 Problem 2: Create a Directory

**Problem:** Write a script that takes a directory name as an argument and creates the directory.

**Solution:**

```
#!/bin/bash
# Create a directory
if [ $# -eq 0 ]; then
    echo "No directory name supplied"
    exit 1
fi

mkdir $1
echo "Directory '$1' created."
```

## 10.3 Problem 3: Check if File Exists

**Problem:** Write a script that checks if a given file exists.

**Solution:**

```
#!/bin/bash
# Check if file exists
if [ -f "$1" ]; then
    echo "File '$1' exists."
else
    echo "File '$1' does not exist."
fi
```

## 10.4 Problem 4: Simple Calculator

**Problem:** Write a script that takes two numbers and an operator (+, -, \*, /) as arguments and performs the corresponding calculation.

**Solution:**

```
#!/bin/bash
# Simple calculator
if [ $# -ne 3 ]; then
    echo "Usage: $0 num1 operator num2"
    exit 1
```

```

fi

num1=$1
operator=$2
num2=$3

case $operator in
    +) result=$(echo "$num1 + $num2" | bc) ;;
    -) result=$(echo "$num1 - $num2" | bc) ;;
    \*) result=$(echo "$num1 * $num2" | bc) ;;
    /) result=$(echo "$num1 / $num2" | bc) ;;
    *) echo "Invalid operator"; exit 1 ;;
esac

echo "Result: $result"

```

## 10.5 Problem 5: Count Lines in a File

**Problem:** Write a script that counts the number of lines in a given file.

**Solution:**

```

#!/bin/bash
# Count lines in a file
if [ -f "$1" ]; then
    lines=$(wc -l < "$1")
    echo "The file '$1' has $lines lines."
else
    echo "File '$1' does not exist."
fi

```

## 11 Intermediate Problems

### 11.1 Problem 6: Backup Script

**Problem:** Write a script to back up a specified directory to a backup directory, adding a timestamp to the backup file.

**Solution:**

```

#!/bin/bash
# Backup script
if [ $# -ne 2 ]; then
    echo "Usage: $0 source_directory backup_directory"
    exit 1
fi

```

```

SOURCE=$1
DESTINATION=$2
DATE=$(date +%Y-%m-%d_%H-%M-%S)

tar -czvf $DESTINATION/backup-$DATE.tar.gz $SOURCE
echo "Backup of '$SOURCE' created at '$DESTINATION/backup-$DATE.
tar.gz'."

```

## 11.2 Problem 7: Find and Replace Text

**Problem:** Write a script to find and replace a specified string in a file.

**Solution:**

```

#!/bin/bash
# Find and replace text in a file
if [ $# -ne 3 ]; then
    echo "Usage: $0 search_string replace_string file"
    exit 1
fi

sed -i "s/$1/$2/g" $3
echo "Replaced all occurrences of '$1' with '$2' in '$3'."

```

## 11.3 Problem 8: List Files with Size

**Problem:** Write a script that lists all files in a directory along with their sizes.

**Solution:**

```

#!/bin/bash
# List files with size
if [ $# -eq 0 ]; then
    directory="."
else
    directory=$1
fi

for file in "$directory"/*; do
    if [ -f "$file" ]; then
        size=$(stat -c%s "$file")
        echo "$(basename "$file"): $size bytes"
    fi
done

```

## 11.4 Problem 9: User Information

**Problem:** Write a script that takes a username as an argument and displays the user ID, group ID, home directory, and shell.

**Solution:**

```
#!/bin/bash
# Display user information
if [ $# -ne 1 ]; then
    echo "Usage: $0 username"
    exit 1
fi

USERINFO=$(getent passwd $1)

if [ -z "$USERINFO" ]; then
    echo "User '$1' not found."
    exit 1
fi

IFS=":" read -r username password uid gid fullname homedir shell
<<< "$USERINFO"

echo "User ID: $uid"
echo "Group ID: $gid"
echo "Home Directory: $homedir"
echo "Shell: $shell"
```

## 11.5 Problem 10: Disk Usage Alert

**Problem:** Write a script that checks the disk usage of a specified directory and sends an alert if it exceeds a certain threshold.

**Solution:**

```
#!/bin/bash
# Disk usage alert
if [ $# -ne 2 ]; then
    echo "Usage: $0 directory threshold_percentage"
    exit 1
fi

directory=$1
threshold=$2

usage=$(du -s "$directory" | cut -f1)
total=$(df "$directory" | tail -1 | awk '{print $2}')
```

```
percentage=$((usage * 100 / total))

if [ $percentage -ge $threshold ]; then
    echo "Warning: Disk usage of '$directory' is at ${percentage}%"
fi
```

## 12 Advanced Problems

### 12.1 Problem 11: Log File Analyzer

**Problem:** Write a script to analyze a log file and report the number of occurrences of each unique IP address.

**Solution:**

```
#!/bin/bash
# Log file analyzer
if [ $# -ne 1 ]; then
    echo "Usage: $0 logfile"
    exit 1
fi

awk '{print $1}' $1 | sort | uniq -c | sort -nr
```

### 12.2 Problem 12: Parallel File Downloads

**Problem:** Write a script to download multiple files in parallel using URLs provided in a text file.

**Solution:**

```
#!/bin/bash
# Parallel file downloads
if [ $# -ne 1 ]; then
    echo "Usage: $0 url_list_file"
    exit 1
fi

while IFS= read -r url; do
    wget "$url" &
done < "$1"
wait
```

## 12.3 Problem 13: Database Backup with Rotation

**Problem:** Write a script to back up a MySQL database and keep only the last 7 backups.

**Solution:**

```
#!/bin/bash
# MySQL database backup with rotation
if [ $# -ne 3 ]; then
    echo "Usage: $0 db_name db_user backup_directory"
    exit 1
fi

DB_NAME=$1
DB_USER=$2
BACKUP_DIR=$3
DATE=$(date +%Y-%m-%d_%H-%M-%S)
BACKUP_FILE="$BACKUP_DIR/$DB_NAME-$DATE.sql"

mysqldump -u $DB_USER $DB_NAME > $BACKUP_FILE
echo "Backup of database '$DB_NAME' created at '$BACKUP_FILE'."

# Keep only the last 7 backups
cd $BACKUP_DIR
ls -t | grep $DB_NAME | sed -e '1,7d' | xargs -d '\n' rm
```

## 12.4 Problem 14: System Health Check

**Problem:** Write a script to check CPU, memory, and disk usage, and send an alert if any of them exceed a certain threshold.

**Solution:**

```
#!/bin/bash
# System health check
CPU_THRESHOLD=80
MEMORY_THRESHOLD=80
DISK_THRESHOLD=80

cpu_usage=$(top -bn1 | grep "Cpu(s)" | sed "s/.*, *\[0-9.\]*%*id.*/\1/" | awk '{print 100 - $1}')
memory_usage=$(free | grep Mem | awk '{print $3/$2 * 100.0}')
disk_usage=$(df / | tail -1 | awk '{print $5}' | sed 's/%//')

if (( $(echo "$cpu_usage > $CPU_THRESHOLD" | bc -l) )); then
    echo "Warning: CPU usage is at ${cpu_usage}%"
fi
```

```

if (( $(echo "$memory_usage > $MEMORY_THRESHOLD" | bc -l) ));
then
    echo "Warning: Memory usage is at ${memory_usage}%"
fi

if (( $disk_usage > $DISK_THRESHOLD )); then
    echo "Warning: Disk usage is at ${disk_usage}%"
fi

```

## 12.5 Problem 15: File Synchronization

**Problem:** Write a script to synchronize files between two directories, ensuring that the destination directory is an exact copy of the source directory.

**Solution:**

```

#!/bin/bash
# File synchronization
if [ $# -ne 2 ]; then
    echo "Usage: $0 source_directory destination_directory"
    exit 1
fi

SOURCE=$1
DESTINATION=$2

rsync -av --delete $SOURCE/ $DESTINATION/
echo "Synchronization complete. '$DESTINATION' is now an exact
copy of '$SOURCE'."

```

## 13 Additional Exercises

### 13.1 Exercise 1: Generate a Random Password

**Problem:** Write a script that generates a random password of a specified length. The password should include a mix of uppercase letters, lowercase letters, numbers, and special characters.

**Hint:** You can use the `tr` and `head` commands to help generate the random password.

### 13.2 Exercise 2: Monitor System Uptime

**Problem:** Write a script that monitors the system uptime and logs it to a file every minute. The script should run in the background and append the uptime information to a file named `uptime.log`.

**Hint:** You can use the `uptime` command and `sleep` in a loop.

### 13.3 Exercise 3: Email Disk Usage Report

**Problem:** Write a script that checks the disk usage of the root directory and sends an email report if the usage exceeds a specified threshold. The email should include the current disk usage percentage.

**Hint:** Use the `df` command to check disk usage and `mail` command to send an email.

### 13.4 Exercise 4: Find Duplicate Files

**Problem:** Write a script that finds and lists duplicate files in a specified directory based on their contents. The script should output the filenames and their paths.

**Hint:** Use checksums (e.g., `md5sum`) to compare file contents.

### 13.5 Exercise 5: Process Manager

**Problem:** Write a script that monitors the system for processes consuming more than a specified amount of CPU or memory and logs these processes to a file named `high_usage.log`. The script should run in the background.

**Hint:** Use the `ps` command to monitor CPU and memory usage.